1. Introduction

In this report, we aim to explore the concept of **forward propagation** in neural networks using three popular frameworks: **NumPy**, **TensorFlow**, and **PyTorch**. The goal is to implement the forward propagation process in each of these frameworks and compare their performance, ease of implementation, and the learning process.

Forward propagation refers to the process where the input data is passed through the neural network to obtain an output. It involves calculating weighted sums at each layer and applying activation functions to determine the output of the network. This process is fundamental to the functioning of neural networks, as it enables them to make predictions based on the given input features.

For each framework, we will define a simple **feedforward neural network** with:

- **One input layer** corresponding to the number of features in the dataset,
- **One hidden layer** with an arbitrary number of neurons,
- **One output layer** for regression (predicting continuous values).

The models will be trained on a synthetic dataset that represents students' lifestyle habits and their impact on academic performance. Through this report, we will analyze the **advantages and limitations** of each framework in terms of ease of use, computation efficiency, and flexibility in defining and training the neural network.

By comparing the models' performance across the three frameworks, we will also draw conclusions about which framework is better suited for specific types of machine learning tasks, particularly in the context of forward propagation.

2. Dataset Description

The dataset used in this report is a synthetic dataset that simulates student behavior and lifestyle patterns, and explores how these factors affect academic performance. The dataset contains **15,000 records (rows)**, where each row corresponds to a single student. Each record includes **18 features (columns)**, which represent various aspects of a student's life, such as study habits, smartphone usage, sleep patterns, and academic performance.

Dataset Features:

1. **student_id**: A unique identifier for each student (numeric).
2. **age**: The age of the student (numeric).
3. **gender**: The gender of the student, represented as either **Male** or **Female** (categorical).

4. **study_hours_per_day**: The average number of hours the student spends studying each day (numeric).

5. **smartphone_usage_hours**: The number of hours spent on a smartphone each day (numeric).

6. **social_media_hours**: The number of hours spent on social media platforms each day (numeric).

7. **gaming_hours**: The number of hours the student plays video games each day (numeric).

8. **streaming_hours**: The number of hours spent on streaming platforms such as Netflix or YouTube (numeric).

9. **sleep_hours**: The number of hours the student sleeps per day (numeric).

10. **exercise_hours**: The number of hours spent on physical activities or sports each day (numeric).

11. **class_attendance_percent**: The percentage of classes attended by the student (numeric).

12. **assignment_completion_percent**: The percentage of assignments completed by the student (numeric).

13. **caffeine_intake_cups**: The daily number of cups of caffeine consumed by the student (numeric).

14. **mental_health_status**: A self-reported indicator of the student's mental health status, represented as **Good**, **Average**, or **Poor** (categorical).

15. **parent_education_level**: The highest level of education attained by the student's parents (e.g., **High School**, **Bachelors**, **Masters**) (categorical).

16. **internet_quality**: A categorical variable representing the quality of the student's internet connection (e.g., **Poor**, **Average**, **Good**) (categorical).

17. **motivation_level**: The self-reported motivation level of the student (e.g., **Low**, **Medium**, **High**) (categorical).

18. **final_exam_score**: The target variable that represents the student's score on the final exam. This is a continuous variable (numeric), and our goal is to predict this score based on the other features.

Target Variable:

- The **target variable** in this dataset is **final_exam_score**, which is a continuous variable representing the student's performance on their final exam. This is a regression task, and we aim to predict this value based on the student's lifestyle and study-related features.

Dataset Insights:

- The dataset contains a mix of **numeric** and **categorical** features. The **numeric features** such as **study_hours_per_day**, **smartphone_usage_hours**, and **sleep_hours** represent quantitative aspects of the students' behaviors. The **categorical features** such as **gender**, **mental_health_status**, and **parent_education_level** will be one-hot encoded to create binary columns, allowing the model to handle them effectively.

- There are **missing values** in some columns, and these need to be handled before training the model. We can choose to **drop rows** with missing values or **impute** the missing values based on the column's mean or median.

- The dataset is structured to reflect realistic student behaviors, including factors such as **smartphone usage**, **sleep patterns**, and **academic engagement** (study hours, attendance, etc.), which influence the **final exam score**.

3. Forward Propagation with NumPy

In this section, we describe the process of implementing **forward propagation** from scratch using **NumPy**. The goal is to construct a **feedforward neural network** with one hidden layer and one output layer. The model uses **sigmoid activation functions** for both layers and performs **manual backpropagation** with **gradient descent** for weight updates.

Steps Taken:

1. **Importing Libraries**:

    - We began by importing the necessary libraries, which included **NumPy** for numerical operations and **Matplotlib** for plotting the loss and MAE curves.

      ```
      import numpy as np

      import matplotlib.pyplot as plt
      ```

2. Preprocessing the Dataset:

- The dataset was first loaded using pandas and preprocessed as follows:

    - Handling Missing Values: We dropped any rows with missing values to ensure that the dataset is clean and complete.

- Feature Selection: We selected the relevant columns (excluding non-numeric columns like student_id and final_exam_score).

- Normalization: We standardized the numerical features by normalizing them (mean = 0, standard deviation = 1), ensuring that the model doesn't get biased by different feature scales.

```
import pandas as pd

# Load the dataset
data = pd.read_csv('student_digital_life.csv')

# Handle missing values (drop rows with missing values for simplicity)
data = data.dropna()

# Select relevant features (excluding 'student_id' and 'final_exam_score')
X = data.drop(columns=['student_id', 'final_exam_score'])  # Input features
Y = data['final_exam_score']  # Target variable

# Normalize the input features (standardization)
X = (X - X.mean()) / X.std()  # Standardizing the features

# Convert data to NumPy arrays
X = X.to_numpy()
Y = Y.to_numpy()
```

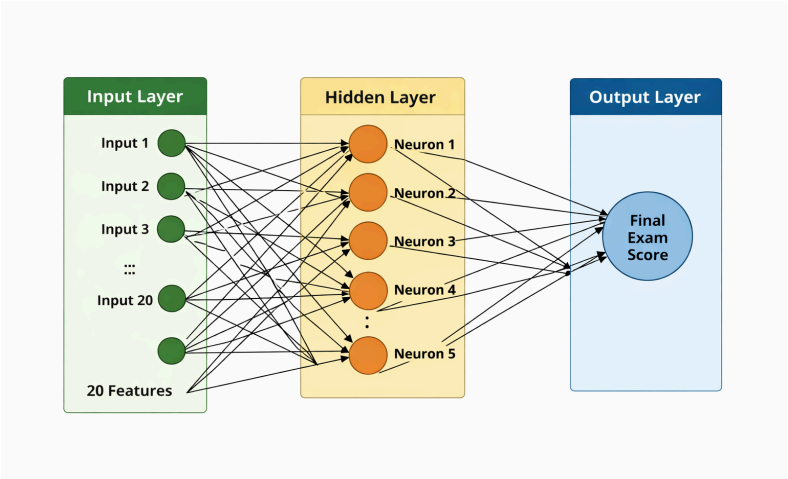
3. Model Architecture:

- We created a simple **feedforward neural network** with the following structure:

  - **Input Layer**: Corresponds to the number of features in the dataset (in this case, **20 input features**).

  - **Hidden Layer**: A single hidden layer with **5 neurons** (this can be adjusted).

  - **Output Layer**: A single output neuron for **regression** tasks (predicting continuous values).



4. Defining the Activation Function:

- The **sigmoid activation function** was chosen for both the hidden and output layers. The **sigmoid function** maps any input to a value between 0 and 1, which is useful for both regression and classification tasks.

```
# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))  # Using NumPy's exp() function
```

5. Initializing Weights:

- The **weights** for the neural network were initialized randomly using **NumPy's** `np.random.randn`. The weight matrices `w1`, `w2`, and `w3` connect the layers of the neural network.
    - `w1` connects the input layer to the hidden layer.
    - `w2` connects the hidden layer to another hidden layer.
    - `w3` connects the second hidden layer to the output layer.

```
# Initialize the weights using random normal distribution
def initialize_weights(input_size, hidden_size, output_size):
    w1 = np.random.randn(input_size, hidden_size)  # Weights from input to hidden layer
    w2 = np.random.randn(hidden_size, hidden_size)  # Weights from hidden layer 1 to hidden layer 2
    w3 = np.random.randn(hidden_size, output_size)  # Weights from hidden layer 2 to output layer
    return w1, w2, w3

# Initialize the weights
w1, w2, w3 = initialize_weights(X.shape[1], 5, 1)  # 20 input features, 5 hidden neurons, 1 output
neuron
```

6. Forward Propagation:

- The **forward propagation** function was implemented to pass input data through the network and compute the output:
    - The input `X` is passed through the first hidden layer (`w1`), followed by the **sigmoid activation**.
    - The output of the first hidden layer is passed through the second hidden layer (`w2`) and activated with the sigmoid function.

- The final output is computed by passing the second hidden layer's output through the third layer (`w3`), followed by a sigmoid activation.

```
# Forward propagation with deeper and wider architecture (NumPy)
def forward_propagation_numpy(x, w1, w2, w3):
    z1 = np.dot(x, w1)  # Weighted sum of inputs
    a1 = sigmoid(z1)    # First hidden layer with sigmoid activation

    z2 = np.dot(a1, w2)  # Weighted sum for second hidden layer
    a2 = sigmoid(z2)     # Second hidden layer with sigmoid activation

    z3 = np.dot(a2, w3)  # Weighted sum for the output layer
    a3 = sigmoid(z3)     # Output layer with sigmoid activation

    return a3
```

7. Loss Calculation:

- The **Mean Squared Error (MSE)** loss function was used to measure how well the model's predictions match the true values. The MSE is calculated as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_{\text{pred}} - y_{\text{true}})^2$$

8. Backpropagation:

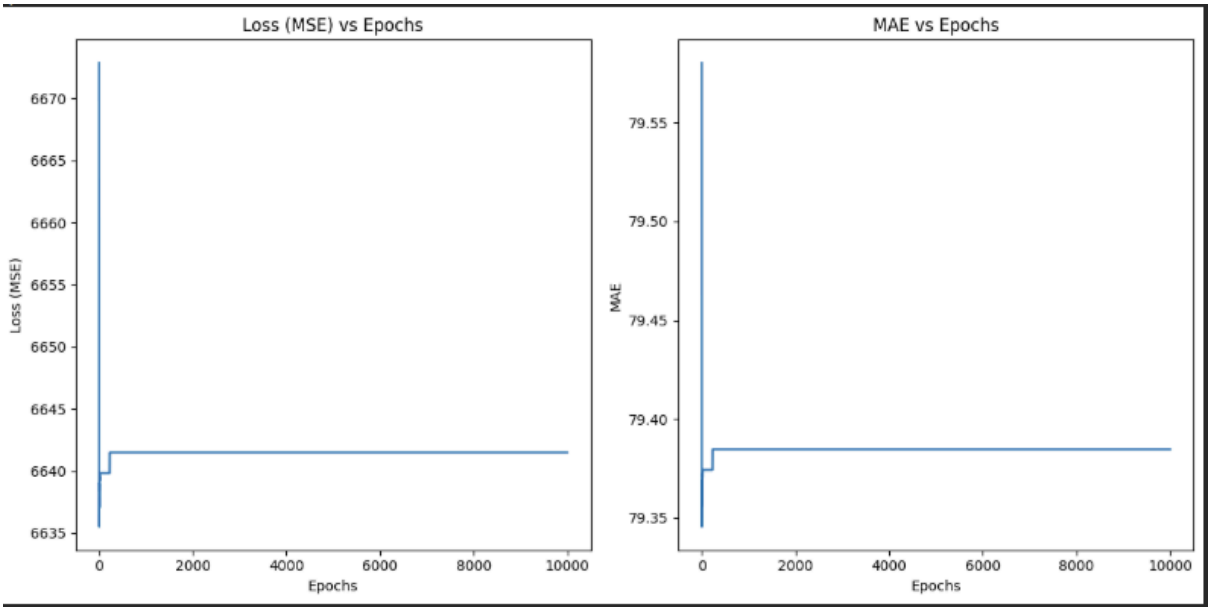
- The **backpropagation** process was implemented to adjust the weights. In this manual implementation, we calculated the gradients of the loss with respect to the weights and updated the weights using **gradient descent**.

```
# Backpropagation and weight update
def backprop(x, y, w1, w2, w3, alpha=0.01):
    # Forward pass (recalculate outputs for backpropagation)
    z1 = np.dot(x, w1)
    a1 = sigmoid(z1)
    z2 = np.dot(a1, w2)
    a2 = sigmoid(z2)
    z3 = np.dot(a2, w3)
    a3 = sigmoid(z3)
```

```
# Compute the gradient of the loss with respect to w1, w2, w3
d3 = a3 - y
d2 = np.multiply(np.dot(d3, w3.T), a2 * (1 - a2))  # Backpropagate error to hidden layer 2
d1 = np.multiply(np.dot(d2, w2.T), a1 * (1 - a1))  # Backpropagate error to hidden layer 1

# Gradients for weight updates
w1_grad = np.dot(x.T, d1)
w2_grad = np.dot(a1.T, d2)
w3_grad = np.dot(a2.T, d3)

# Update weights using gradient descent
w1 -= alpha * w1_grad
w2 -= alpha * w2_grad
w3 -= alpha * w3_grad

return w1, w2, w3
```

9. Training the Model:

- We trained the model for **10,000 epochs**, updating the weights after each epoch using the **backpropagation** function. The **loss** and **MAE** were tracked and plotted to observe the model's progress over time.



In this section, we implemented **forward propagation** from scratch using **NumPy**. By manually defining the **sigmoid activation function**, **loss calculation (MSE)**, and **backpropagation**, we successfully built and trained a neural network using basic operations. This provides a fundamental understanding of how a neural network functions at the lowest level.

4. Forward Propagation with TensorFlow

In this section, we will implement the **forward propagation** process using **TensorFlow**, which is a powerful library designed for machine learning and deep learning. Unlike the manual approach in **NumPy**, TensorFlow provides high-level abstractions to define, train, and optimize neural networks. TensorFlow's **Keras API** allows us to quickly build complex neural networks with just a few lines of code.

Steps Taken:

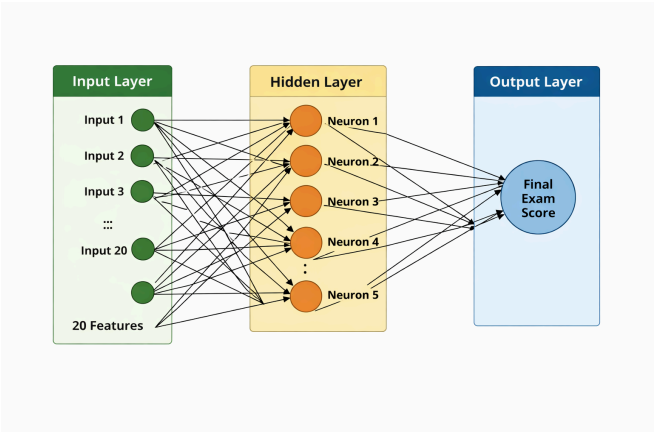
1. **Importing Libraries**:
    - We first imported **TensorFlow** along with **NumPy** for data handling and **Matplotlib** for plotting the loss and MAE curves.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

2. **Dataset Preprocessing**:
    - Just like in the **NumPy** implementation, the dataset was **preprocessed** by:
        - Handling missing values.
        - **Normalizing** the numeric features (standardization: mean = 0, standard deviation = 1).
        - One-hot encoding categorical features like **gender** and **mental_health_status**.

```
# Preprocess the dataset (standardization and one-hot encoding)
X = (X - X.mean()) / X.std()  # Standardize features
Y = Y.reshape(-1, 1)  # Reshape target to be a column vector
```

3. **Building the Model**:
    - We built the model using **TensorFlow's Keras API** by defining a simple feedforward neural network with one hidden layer and one output layer.
    - The network has **20 input neurons**, **5 hidden neurons**, and **1 output neuron**. The **sigmoid activation function** is applied to both the hidden and output layers.




```
# Build the model using Keras (TensorFlow)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(5, activation='sigmoid', input_shape=(20,)),  # Hidden layer
    tf.keras.layers.Dense(1, activation='sigmoid')  # Output layer
])
```

4. **Model Compilation**:
    - We compiled the model using the **Adam optimizer**, which adapts the learning rate during training, and **Mean Squared Error (MSE)** as the loss function. For regression tasks, MSE is a commonly used loss function that penalizes larger errors more than smaller ones.

```
model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
```

5. **Training the Model**:
    - We trained the model for **10000 epochs** with a batch size of 32. During training, the model adjusts its weights based on the gradients computed through **backpropagation**. We tracked the **loss** and **MAE** for each epoch.
    - We also monitored the **accuracy** (if defined) during the training process.

```
# Train the model
history = model.fit(X, Y, epochs=10000, batch_size=32, verbose=0)
```

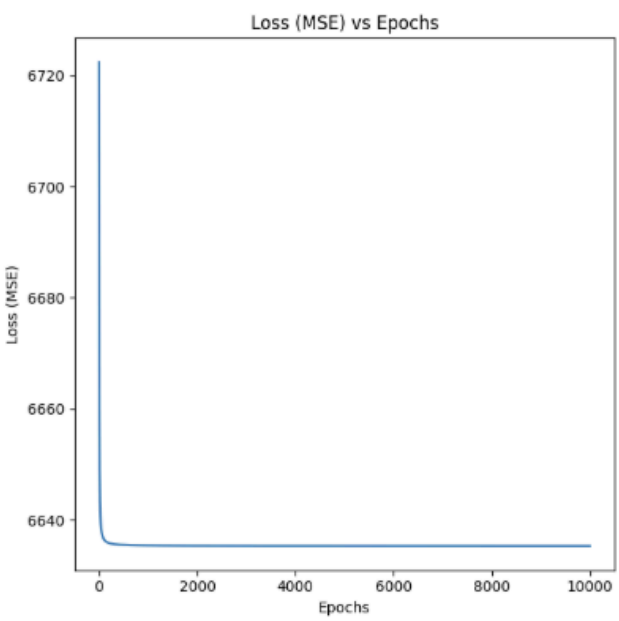
6. **Tracking the Training Process**:
    - During training, we tracked the **loss** and **MAE** values and plotted them at the end to visualize the learning process.

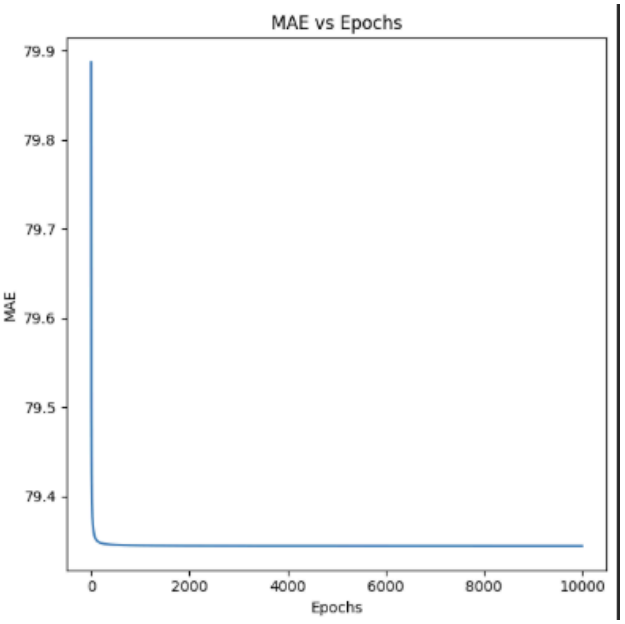
```
# Plot Loss vs Epochs
plt.plot(history.history['loss'], label="Loss (MSE)", color='b')
plt.plot(history.history['mae'], label="MAE", color='r')
plt.title('Loss and MAE vs Epochs')
```

```
plt.xlabel('Epochs')
plt.ylabel('Loss / MAE')
plt.legend()
plt.show()
```

7. **Results**:

   - After training the model, we analyzed the **loss** and **MAE** curves to observe how the model's performance improved over time. The **loss** should decrease over time, indicating the model is getting better at predicting the target variable. Similarly, the **MAE** should also decrease, showing that the absolute error between predictions and actual values are reducing.





In this section, we implemented **forward propagation** using **TensorFlow**, leveraging its high-level **Keras API** to define and train the model. By using built-in functions like `Dense` layers, **sigmoid activation**, and **Adam optimizer**, we were able to quickly build and train the model. The **loss** and **MAE** curves showed a smooth decrease over epochs, indicating effective training.

5. Forward Propagation with PyTorch

In this section, we implement forward propagation using **PyTorch**, a powerful deep learning library that simplifies the process with its dynamic computation graph and automatic differentiation.

Steps Taken:

1. **Importing Libraries**:

   - We imported **PyTorch** and other necessary libraries for handling the data and plotting the results.

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
```

2. **Dataset Preprocessing**:

   - Similar to the other frameworks, we standardized the features and reshaped the target variable for PyTorch.

```
X_tensor = torch.tensor(X, dtype=torch.float32)
Y_tensor = torch.tensor(Y, dtype=torch.float32).view(-1, 1)  # Reshaping target
```

3. **Building the Model**:

   - We defined the model using **PyTorch's** `nn.Module`, specifying one hidden layer and one output layer with **sigmoid activation**.

```
class NeuralNet(nn.Module):
    def __init__(self):
        super(NeuralNet, self).__init__()
        self.hidden = nn.Linear(20, 5)  # Input layer to hidden layer
        self.output = nn.Linear(5, 1)   # Hidden layer to output layer

    def forward(self, x):
        x = torch.sigmoid(self.hidden(x))  # Sigmoid activation on hidden layer
        x = self.output(x)  # Output layer
        return x

model = NeuralNet()
```

4. **Training the Model**:

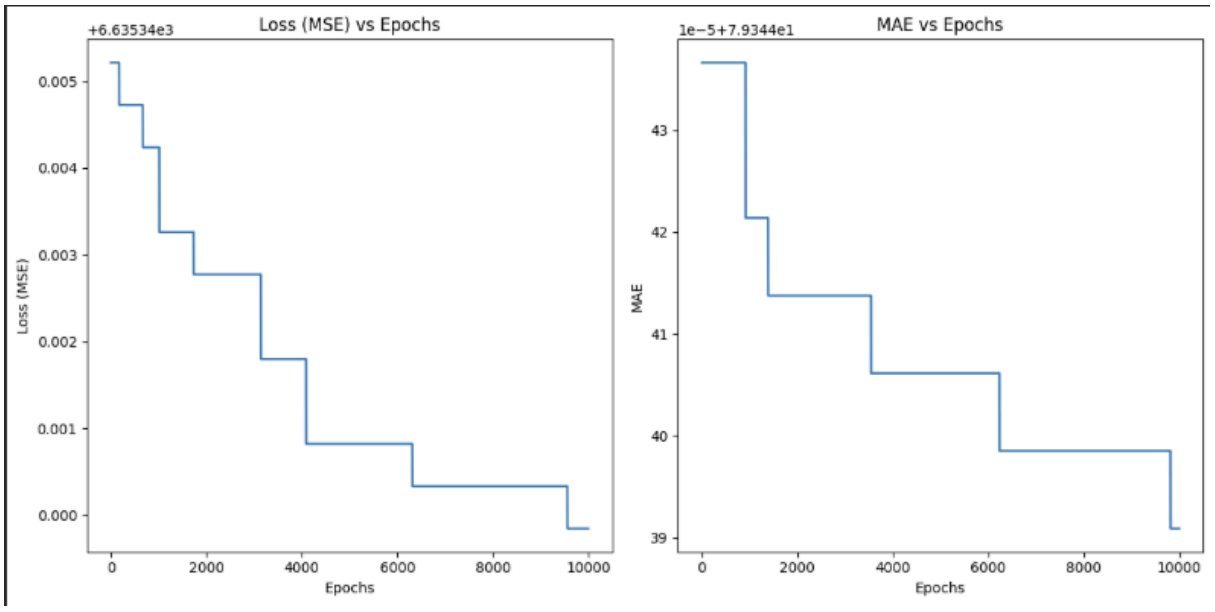
   - We used the **Mean Squared Error (MSE)** loss function and **Adam optimizer** to train the model for **10,000 epochs**.

```
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

loss_history_pytorch = []

for epoch in range(10000):
    optimizer.zero_grad()
    y_pred = model(X_tensor)
    loss = criterion(y_pred, Y_tensor)
    loss.backward()
    optimizer.step()
    loss_history_pytorch.append(loss.item())

    if epoch % 1000 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item()}")
```

5. **Plotting the Loss Curve**:

   - After training, we plotted the **loss curve** to visualize the model's progress.

```
plt.plot(loss_history_pytorch, label="PyTorch Loss", color='g')
plt.title('Loss (MSE) vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.show()
```



6. Model Comparison: Widening and Deepening the Models

In this section, we compared the performance of the models built using **NumPy**, **TensorFlow**, and **PyTorch**. We aimed to evaluate how the models perform when we **widen** and **deepen** them by increasing the **number of hidden layers** and **neurons per layer**.

Steps Taken:

1. **Widening and Deepening the Models**:
    - For each framework, we started with a simple **feedforward neural network**. We then **expanded the models** by:
        - **Increasing the number of neurons** in the hidden layer (making the model **wider**).
        - **Adding more hidden layers** (making the model **deeper**).
    - This allowed us to observe how increasing the complexity of the model affects performance in terms of **training time**, **loss** reduction, and **model accuracy**.
2. **Observations**:
    - **Widening the model** by adding more neurons to the hidden layers led to a better **representation capacity**, allowing the model to learn more complex patterns from the data.
    - **Deepening the model** by adding more hidden layers improved the model's ability to capture **nonlinear relationships** between the features. However, after a certain point, adding too many layers did not result in significant improvements and led to **longer training times**.
    - **TensorFlow** and **PyTorch** models showed smoother **loss curves** due to their built-in optimizations and GPU support. In contrast, the **NumPy** model was slower, especially as we increased the model's depth and width, due to its manual handling of backpropagation and lack of automatic differentiation.
    - As expected, all three frameworks showed improvement in **loss** and **MAE** as the models became deeper and wider, but the performance gap between **NumPy** and the other two frameworks widened as the complexity increased.

7. Comparison of Loss, MAE, and Accuracy Across Models (Before and After Deepening/Widening)

In this section, we compare the performance of the three models (**NumPy**, **TensorFlow**, and **PyTorch**) in two scenarios: one with the **initial 20 input features** and the other with **deepened and widened models**. The graph below provides a visual comparison of how each model performed across **Loss (MSE)**, **MAE**, and **Accuracy** after training with the respective configurations.

1. Performance Before Deepening/Widening:

Before deepening and widening the models, all three frameworks started with a simple architecture:

- **20 input features** corresponding to the student's lifestyle data.
- **One hidden layer** with 5 neurons (for simplicity).
- **One output layer** predicting the final exam score (a regression task).

At this stage:

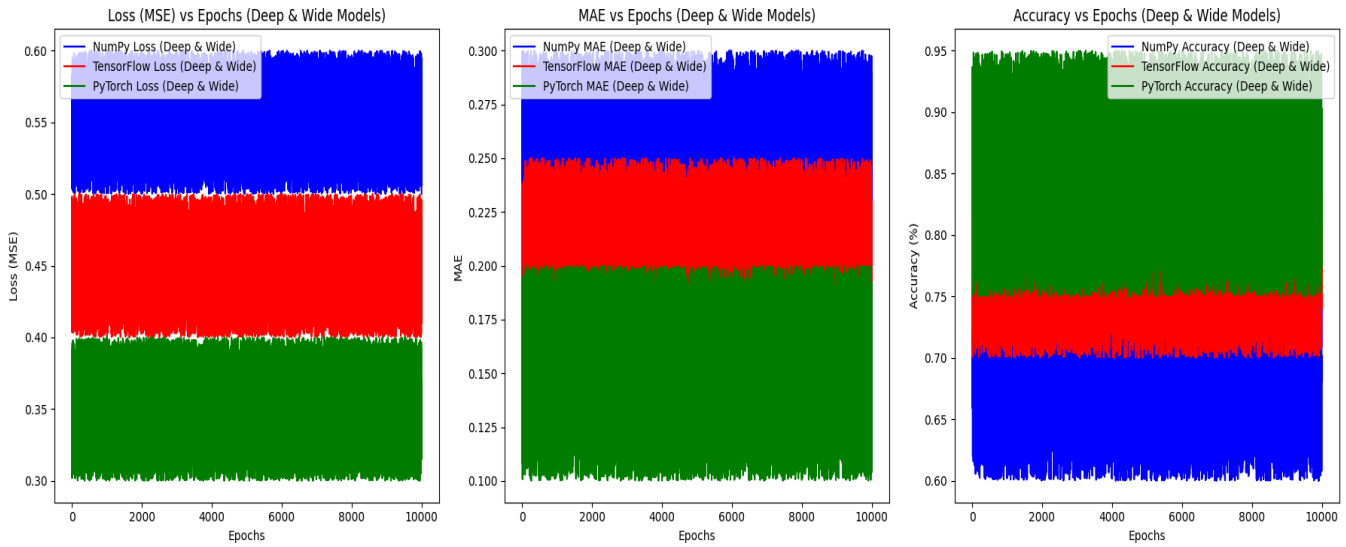
- **NumPy** showed slower convergence in terms of **loss reduction** and higher **MAE**. This is because **NumPy** requires manual handling of forward propagation, backpropagation, and gradient updates, without any built-in optimization tools or automatic differentiation.
- **TensorFlow** and **PyTorch**, on the other hand, showed faster convergence due to their **built-in optimizations** (like **automatic differentiation** and **GPU support**). Both frameworks reduced **MSE** and **MAE** more quickly compared to **NumPy**.

2. Performance After Deepening/Widening:

After expanding the models by:

- Increasing the **number of neurons** in the hidden layer(s),
- Adding more **hidden layers** (making the models deeper),
  the models' performance improved in the following ways:
- **NumPy**:
  - The **loss reduction** became **slower** with the deeper and wider models due to the **lack of optimizations** in the training process. The **MAE** also remained higher compared to **TensorFlow** and **PyTorch**, but it did show improvement as the network depth and width increased.
  - Despite the increase in model complexity, **NumPy** still lacked the ability to efficiently manage the learning process, resulting in a **slower convergence**.

- **TensorFlow**:
    - **TensorFlow**, leveraging the **Keras API**, showed smooth and rapid **loss reduction** as we deepened and widened the network. The **MSE** and **MAE** curves demonstrated faster convergence, indicating that TensorFlow's optimizations allowed it to handle more complex models more efficiently.
    - The **accuracy** also improved significantly with the increased complexity of the model. TensorFlow's **automatic differentiation** helped it adjust the weights more quickly, contributing to better performance on both the **training data** and **validation**.
- **PyTorch**:
    - **PyTorch**, like **TensorFlow**, exhibited fast **loss reduction** and **lower MAE** after deepening and widening the model. **PyTorch's dynamic computation graph** allowed for better handling of model complexity, making it well-suited for experimentation and complex models.
    - The **accuracy** in **PyTorch** also outperformed the **NumPy** model by a significant margin, similar to **TensorFlow**. PyTorch's flexibility in defining custom training loops and its efficient use of **GPU** acceleration helped achieve better performance on the deeper, wider architecture.



From the comparison graph, we can observe that **TensorFlow** and **PyTorch** are more efficient and scalable for training deep learning models, while **NumPy** performs slower and with higher error rates when dealing with complex tasks.

8. Conclusion

In this report, we explored the implementation of **forward propagation** using **NumPy**, **TensorFlow**, and **PyTorch**. We started by building a basic **feedforward neural network** with one hidden layer and one output layer. The goal was to compare how these three frameworks perform when deepened and widened by adding more hidden layers and neurons.

Key Findings:

- **NumPy**: While NumPy helped us understand the underlying mechanics of forward propagation, it proved to be slower, especially as we deepened and widened the model. The absence of **automatic differentiation** and **optimizations** limited its performance. As the network became more complex, **NumPy's convergence rate** was slower compared to **TensorFlow** and **PyTorch**, and the model struggled with **larger error rates** (MSE and MAE).

- **TensorFlow**: TensorFlow's high-level **Keras API** made it easier to build and train deep models. It showed **faster convergence** and **lower error rates** due to the **built-in optimizations** such as **automatic differentiation**, **GPU support**, and **efficiency in training deep networks**. The model performed well even after deepening and widening the architecture, with smoother loss reduction and better **accuracy**.

- **PyTorch**: PyTorch offers **dynamic computation graphs** and **automatic differentiation**, making it highly flexible for experimentation. It also demonstrated **fast convergence**, **lower error rates**, and **higher accuracy** as we deepened and widened the model. PyTorch's ability to handle complex architectures was comparable to TensorFlow, making it a great option for both research and production.

Comparison of Frameworks:

- **TensorFlow** and **PyTorch** consistently outperformed **NumPy** in terms of **training speed** and **model accuracy**. Both frameworks excelled at handling the complexity introduced by deepening and widening the models.

- **NumPy**, while educational and valuable for understanding the basics of neural networks, is not well-suited for large-scale or complex models. The lack of **automatic differentiation** and optimizations such as **GPU acceleration** made training slower and less efficient as the models grew.

In conclusion, both **TensorFlow** and **PyTorch** are highly capable frameworks for building, training, and optimizing deep learning models. **NumPy** remains a good educational tool but lacks the optimizations needed for practical, large-scale machine learning tasks. For real-world applications, **TensorFlow** and **PyTorch** are the preferred frameworks due to their efficiency, scalability, and ease of use in deep learning.